Structural Inspection Path Planning for Autonomous Drone Navigation

Tianhao Liang

1 Problem Statement and Objectives

Autonomous drone-based structural inspection requires visiting a set of predefined viewpoints while avoiding obstacles in 3D space. This coursework addresses the *Structural Inspection Path Planning Challenge*: based on a scenario with n viewpoints (each with a position and camera yaw angle), m obstacles, and a drone start position, the drone could plan an efficient flight route (shortest distance) that visits all viewpoints and detect the ArUco marker while no collision happening.

The objectives are threefold: (1) minimise total flight distance and time by optimising the visit order; (2) use path planning to avoid obstacles; (3) verify mission completion by detecting ArUco markers placed at each viewpoint. The solution is validated through four scenarios of increasing complexity (5–20 viewpoints, 1–10 obstacles) in the Gazebo simulation environment.

2 Methodology

The system is implemented as two modules: a planning module (`planner.py`) that runs before take-off, and a mission execution module (`mission_scenario.py`) that interfaces with the drone through ROS 2.

2.1 Scenario Parsing and Collision Detection

Obstacles are represented as axis-aligned bounding boxes (AABBs). A ray–AABB intersection test (Slab Method) determines whether a straight-line between two points passes through any obstacle, enabling the construction of a collision-aware distance matrix.

2.2 Waypoint Sequencing: TSP Optimisation

The viewpoint visit order is formulated as a classic Travelling Salesman Problem (TSP). A collision-aware distance matrix is constructed where edges passing through obstacles receive a $2\times$ penalty. The $2\times$ distance penalty acts as a heuristic to encourage the selection of straight-line paths. This approach avoids the prohibitive computational cost of calculating A* paths for all potential node pairs while effectively prioritizing safe and direct flight paths. The TSP is solved using the `python_tsp` library: exact dynamic programming is applied for the $n \leq 15$ nodes, while a 2-opt local search is used for larger instances with more than 15 nodes to guarantee the computation time is acceptable.

A nearest-neighbour heuristic is implemented in `planner_tools.py` for comparison of methods; the main mission uses only the exact/2-opt solver. The exact/2-opt approach was chosen for flight because it guarantees optimality for small instances and gives near-optimal solutions for larger ones.

2.3 Motion Planning: 3D A* with Path Simplification

For each consecutive pair of viewpoints in the TSP order, the planner first checks whether a direct straight-line and collision-free path exists. If so, the drone flies directly; otherwise, a 3D A* search finds a collision-free path.

The A* algorithm works as follows. The workspace is discretised into a 3D grid (1.0 m resolution). Each node has a cost g (actual cost from start) and an estimate $f = g + h$ where h is the Euclidean distance to the goal (admissible heuristic). The algorithm maintains an open set of nodes to expand, ordered by f. It repeatedly takes the node with the smallest f, marks it closed, and expands its 26-connected neighbours (same and adjacent cells in x, y, z). For each neighbour that is inside the grid, outside the inflated obstacles (0.8 m buffer), and not already closed, it computes the tentative g via the current node; if this is lower than the neighbour's existing g, it updates the neighbour and records the current node as its predecessor, then adds the neighbour to the open set. When the goal node is expanded, the path is recovered by backtracking along the recorded predecessors. The resulting grid path is then simplified: any intermediate waypoint is removed if the straight segment from the previous to the next waypoint is collision-free, reducing the number of segments sent to the flight controller.

Why A* rather than RRT or PRM? The path planning method must support verification and fair comparison: we need repeatable, optimal paths so that metrics (distance, time) are comparable across runs . A* on a discrete 3D grid provides exactly that: (i) it is **deterministic** (same start/goal and obstacles always yield the same path); (ii) it is **optimal** for the given resolution . By contrast, RRT and PRM are sampling-based —different runs give different paths and distances and may not be the optimal. So for low-dimensional (2D/3D) spaces with simple environment, A* is a standard and well-understood choice, the run time is totally acceptable for real-time tasks.

2.4 ArUco Marker Verification

Task completion is verified by detecting ArUco markers (DICT_5X5_1000 dictionary) placed at each viewpoint using the drone's onboard HD camera. A key engineering challenge was integrating marker detection without disrupting flight stability. The final architecture uses a dedicated ROS 2 node with its own single-threaded executor running on a separate Python thread. Camera subscriptions are created only when the drone arrives at a viewpoint and destroyed immediately after detection (or timeout), ensuring zero overhead during flight. This architecture prevents the erratic drone behaviour caused by CPU resource contention.

2.5 Integration into the Gazebo Simulation Environment

The drone simulation (Gazebo) is launched with a scenario. The mission module loads the scenario, gets from the planner the visit order (TSP) and the list of waypoints that avoid obstacles (A* where needed), then runs the mission: the drone takes off, flies to each waypoint (turning at viewpoints so the camera faces the marker), and verifies the ArUco marker at each viewpoint. Metrics are logged after each run for comparison and verification.

3 Results and Discussion

All four scenarios were completed successfully in Gazebo. Table 1 summarises the key metrics.

Table 1: Performance metrics across all four test scenarios.

Metric	**Scenario 1**	**Scenario 2**	**Scenario 3**	**Scenario 4**
Viewpoints / Obstacles	10 / 1	10 / 5	20 / 1	5 / 10
Planning time (s)	0.02	0.99	0.09	0.85
Baseline execution time (s)	176.83	246.81	266.85	85.3
Optimised execution time (s)	81.72	99.34	128.85	73.01
Viewpoints visited	10/10	10/10	20/20	5/5
ArUco markers verified	10/10	10/10	20/20	5/5
Optimised distance (m)	60.50	64.61	82.66	55.19
Baseline distance (m)	125.93	127.40	175.80	54.09
Distance improvement	52.0%	49.3%	53.0%	-2.0%
Average speed (m/s)	0.74	0.65	0.64	0.76

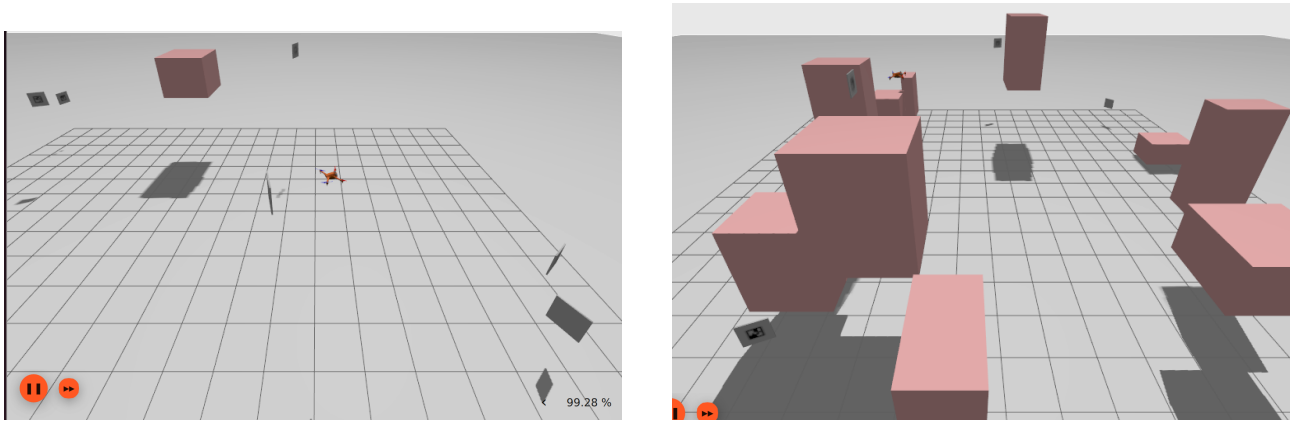
3.1 Comparison of Planning Methods

Table 2 reports the **planned route distance** (sum of segment lengths before execution) and **planning time** for each method across all four scenarios.

Table 2: Comparison of visit-order methods: planned distance (m) and planning time (ms).

Method	**Metric**	**Sc. 1**	**Sc. 2**	**Sc. 3**	**Sc. 4**
Baseline (YAML order)	Distance (m)	126.1	149.8	175.5	86.6
	Planning (ms)	<1	<1	<1	<1
TSP (exact / 2-opt)	Distance (m)	70.6	73.6	91.4	79.2
	Planning (ms)	26.5	17.5	4.9	0.1
TSP nearest-neighbour	Distance (m)	73.2	80.9	99.8	83.4
	Planning (ms)	<1	<1	<1	<1

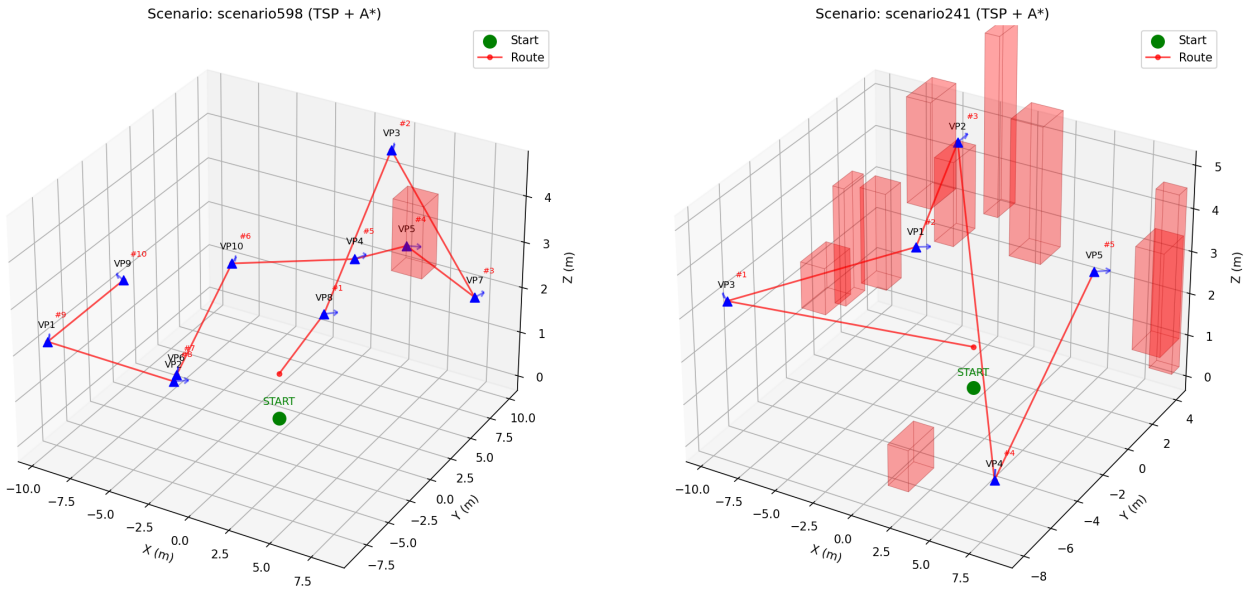
TSP (exact/2-opt) reduces planned distance by about 44–53% versus baseline in Scenarios 1–3; in Scenario 4 the baseline order is already reasonable, so the gain is smaller. Nearest-neighbour is very fast but yields longer routes than exact/2-opt.



(a) Scenario 1 (10 VP, 1 obstacle)

(b) Scenario 4 (5 VP, 10 obstacles)

Figure 1: Gazebo simulation environments for Scenarios 1 and 4.



(a) Scenario 1: all direct paths

(b) Scenario 4: A* detour segments

Figure 2: Planned flight paths. Direct segments shown as straight lines; A* detours shown as multi-segment paths around obstacles.

3.2 TSP Optimisation Effectiveness

For Scenarios 1–3, TSP optimisation reduced total flight distance by approximately **50%** compared to the baseline YAML ordering. Scenario 4 showed a marginal 2% increase because with only 5 viewpoints, the original ordering was already near-optimal, and the collision-aware penalty slightly altered the route to favour obstacle avoidance over distance.

An interesting result is observed in Scenario 4, where the distance improvement is slightly negative (-2.0%), yet the execution time is significantly reduced from 85.3 s (baseline) to 73.01 s. This discrepancy highlights a key advantage of the collision-aware penalty: the baseline YAML order, while shorter in Euclidean distance, likely forced the drone through heavily obstructed paths requiring complex, multi-segment A* detours. In contrast, the optimized route prioritized marginally longer but "clearer" segments, enabling a higher average speed and more efficient flight. This demonstrates that

in dense obstacle environments (10 obstacles), reducing sharp directional changes and intermediate pausescan be more critical to mission performance than raw distance minimization.

3.3 Collision Avoidance

Scenarios 1–3 required no detour waypoints as all direct paths between consecutive viewpoints were obstacle-free. Scenario 4, with 10 densely packed obstacles a, required A* to plan 3 detour segments. The 0.8 m safety buffer enables reliable passage even with controller position tracking errors.

3.4 ArUco Verification

ArUco markers were successfully detected at all 45 viewpoints across all four scenarios (100% verification rate). This demonstrates the robustness of the on-demand subscription architecture: by isolating camera processing onto a dedicated ROS 2 executor thread and subscribing only during viewpoint stops, marker detection operates reliably without interfering with flight control.

3.5 Limitations and Potential Improvements

The PX4 flight controller occasionally follows curved trajectories rather than straight lines when large yaw adjustments ($> 90°$) coincide with position changes. While this does not prevent mission completion, it increases actual flight distance beyond the planned optimum. And the 1.0 m grid resolution limits the drone's ability to navigate through narrow spaces. However, significantly increasing the resolution would lead to a computational explosion in 3D, causing unacceptable delays in path planning. Additionally, the current planning results in seldom stop-and-go maneuvers and sharp turns, which reduce flight efficiency and increase energy consumption.

4 Conclusions

This work presented an integrated path planning system for autonomous drone-based structural inspection. The combination of TSP-based visit ordering and A* collision-free motion planning achieved a 50% reduction in flight distance for scenarios with 10–20 viewpoints, while maintaining zero collisions across all test environments. ArUco marker detection provided reliable visual verification of task completion at all viewpoints.

The approach is effective for structured inspection scenarios with static, known obstacles and moderate numbers of viewpoints. Future work will focus on balancing grid resolution and computational load to support more complex environments. Furthermore, incorporating trajectory optimization could help reduce pauses and turning maneuvers, thereby improving overall flight energy efficiency and performance.